

# Data Science

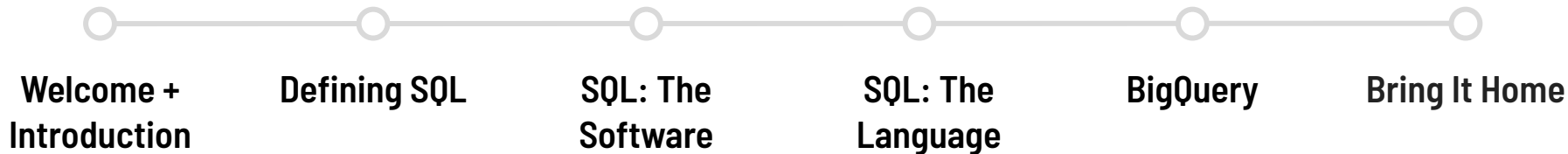
## SQL I – Intro to Queries and BigQuery

Data Science Fundamentals



GENERAL ASSEMBLY

# Lesson roadmap



# Learning objectives



1

Explain what SQL is and understand what a relational database is.

2

Use Google BigQuery to access data via SQL.

3

Write and execute a SQL query, including SELECT, FROM, LIMIT, and DISTINCT.

4

Use GROUP BY in SQL to group rows of data, and summarize those rows.

5

Practice writing aggregate functions: MIN, MAX, SUM, AVG, and COUNT.

# Defining SQL

# What is SQL?

Structured Query Language (SQL) is:

- Pronounced either “sequel” or “ess-kyu-ell”.
- Both the name of a language and a database software.



# SQL: The Software

# SQL The Database Software

A **SQL server** is a server that accepts **SQL queries** and stores data in a **SQL database**.

A SQL database is made up of many **SQL tables**.

But we already know how to store data in files!

**What's the advantage of having data stored in a SQL database?**

# SQL advantages

A SQL database is...

- **Relational**
  - A relational database organises tables so they relate to one another via **keys**. By **joining** tables on these shared keys, you can easily query and retrieve precisely the information you need from multiple tables.

# SQL advantages

A SQL database is...

- **Relational**
  - A relational database organises tables so they relate to one another via **keys**. By **joining** tables on these shared keys, you can easily query and retrieve precisely the information you need from multiple tables.
- **Secure** (as in, “foolproof”)
  - **DBAs (database administrators)** can set up permissions structures in SQL so that the intern doesn’t accidentally delete important tables.



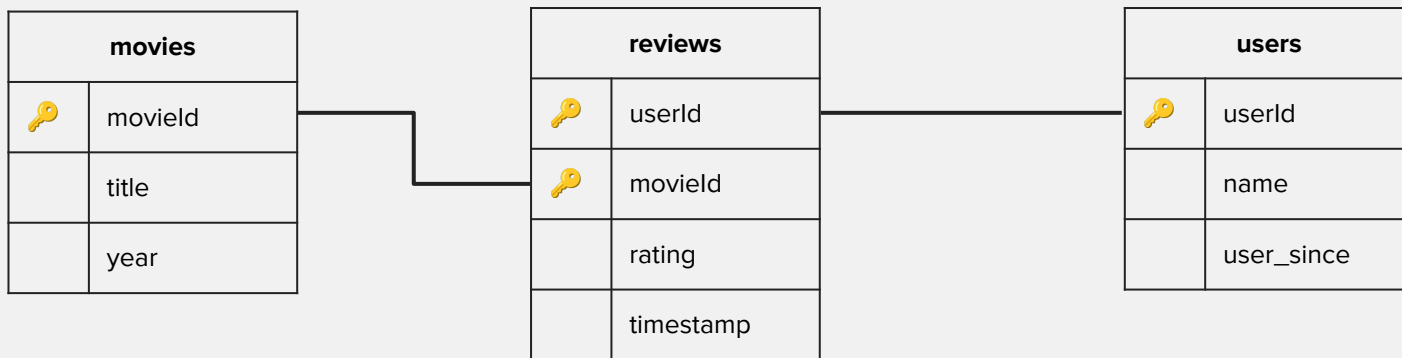
# SQL ENTITY RELATIONSHIP DIAGRAMS (ERDs)

ERDs enable you to describe the relationship between tables in your database.

Again, if things are set up properly, an **entity relationship diagram (ERD)** can be made (or generated!).

This is a graph that describes the relationships between tables in your SQL DB.

# ERD Example



# SQL Join Example

| userId | movield | rating | timestamp    |
|--------|---------|--------|--------------|
| 1      | 1       | 5      | 1/1/22 1:00  |
| 2      | 1       | 4      | 1/1/22 2:00  |
| 2      | 2       | 2      | 1/2/22 11:00 |

| userId | name         | user_since |
|--------|--------------|------------|
| 1      | Tim Book     | 2/28/2019  |
| 2      | Katie Sylvia | 8/17/2020  |

| movield | title    | year |
|---------|----------|------|
| 1       | Shrek    | 2001 |
| 2       | The Hulk | 2008 |

| userId | movield | rating | timestamp    | name         | user_since | title    | year |
|--------|---------|--------|--------------|--------------|------------|----------|------|
| 1      | 1       | 5      | 1/1/22 1:00  | Tim Book     | 2/28/2019  | Shrek    | 2001 |
| 2      | 1       | 4      | 1/1/22 2:00  | Katie Sylvia | 8/17/2020  | Shrek    | 2001 |
| 2      | 2       | 2      | 1/2/22 11:00 | Katie Sylvia | 8/17/2020  | The Hulk | 2008 |

# SQL Advantages

A SQL database is...

- ❑ **Redundant** (as in “safe”)
  - DBAs can set SQL DBs to be periodically backed-up in case something goes wrong.
  - DBs are often stored in redundant hardware on cloud servers so that if one goes down or is destroyed, data aren't lost.

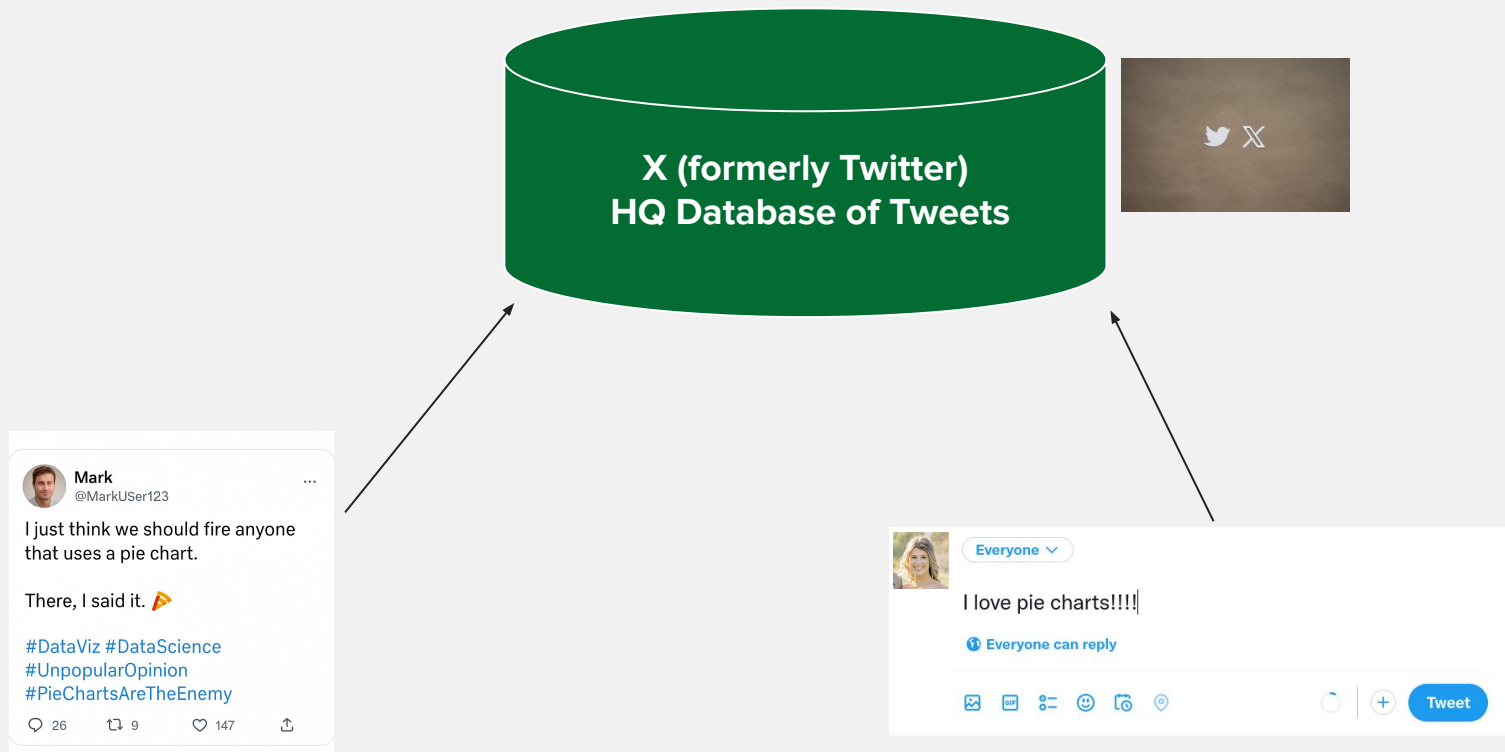
# SQL Advantages

A SQL database is...

## □ Parallel

- SQL DBs are designed so that multiple users can edit and query simultaneously.
- SQL can also **lock** rows in a table when one person is editing it so another person doesn't also try to edit it.
- DBs are often large in terms of storage and powerful in terms of CPU strength. Depending on the architecture, SQL can handle large, distributed calculations!

# Parallel SQL Operations



# SQL: The Language

# SQL is also the programming language that is used to query a SQL database!

- It is a **domain-specific**, **declarative** language
  - domain-specific = only good for one domain (data stuff)
  - declarative = specify only *what* is to be done, not *how*
- As far as languages go, it is very old (1970s!)
- It is very easy to pick up. You will be able to handle 99% of queries after a few lessons

# How is the SQL Language used?

- Historically it has been used as a **standalone language** to extract data from a database to then be used in reports and elsewhere
- It is also thriving as a language invoked via **glue languages** such as Python, R, JavaScript and Ruby
- SQL is the industry-standard for extracting data from relational databases



# The SQL Language is not just one language...

- While there is a “SQL standard”, different versions of SQL do things pretty differently sometimes.
- These differences aren’t worth memorizing... but they’re worth Googling when you encounter something that should work but doesn’t!

# There are many “dialects” of SQL

Here are just a few:

- ❑ **Postgres**: Open source, probably the most common
- ❑ **MySQL**: Also open source, but distributed by Oracle
- ❑ **Transact-SQL (T-SQL)**: SQL built by Microsoft for MS SQL Server
- ❑ **SQLite3**: A light and portable version of SQL often used to build small examples (often in Python). Contained in the Python stdlib!
- ❑ **HiveQL (HQL)**: Ok technically not a SQL dialect, but it looks pretty identical to SQL and actually powers the backend of **Spark**!

# BigQuery

# Google BigQuery

Google BigQuery is a **powerful cloud-based data warehouse and analytics platform**. It is designed to handle large, distributed datasets at scale, allowing users to store and analyze massive amounts of data efficiently.

**SQL is used to interact with BigQuery**. Specifically, BigQuery supports Standard SQL dialect (also known as GoogleSQL) for querying data.



Google  
Big Query

# Why Choose BigQuery?

More importantly for us today, Google provides many example databases for us to use for free! And it is a commonly used SQL solution for Data Scientists

But other advantages of BigQuery include:

- ❑ Fully managed serverless data warehouse - No infrastructure to manage
- ❑ Scalability - Handles petabyte-scale datasets effortlessly
- ❑ Speed - Query terabytes in seconds, petabytes in minutes
- ❑ Cost-effective - Pay only for storage and processing used
- ❑ Built-in AI/ML capabilities - Create and deploy ML models with SQL
- ❑ Real-time analytics - Supports streaming data and real-time querying
- ❑ Multi-cloud and multi-format - Works with various data types and cloud platforms
- ❑ Security and compliance - Strong encryption and access controls
- ❑ Easy integration - Connects seamlessly with other Google Cloud services
- ❑ Familiar SQL interface - Use standard SQL for querying and analysis

# AWS and Azure have similar products to BigQuery



Google  
Big Query



Azure  
Synapse  
Analytics



**amazon**  
REDSHIFT

# BigQuery Set Up

- Let's set up!
- You will need a Google account.
- Open **bigquery-setup-guide.md**



# In SQL, the order of clauses matters

Your SQL clauses have to be in a specific order, otherwise you'll throw an exception.

1. **SELECT** - What columns do you want?
2. **FROM** - Which table?
3. **WHERE** - What conditions must be met? \*haven't covered this yet\*
4. **GROUP BY** - What groups do you want to see?
5. **HAVING** - What conditions must be met for groups? \*haven't covered this yet\*
6. **ORDER BY** - How do you want to order the results?
7. **LIMIT** - How many rows do you want to return?

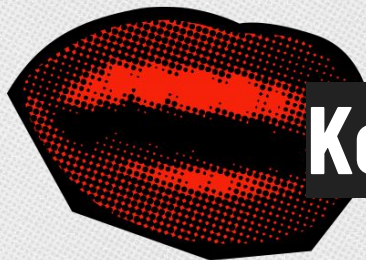
Mnemonic to remember the order: "Smelly Feet Will Give Horrible Odors, Lingeringly"

# Google BigQuery

Let's walk through the repo!



# Bring It Home



# Key takeaways



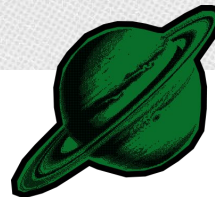
## A SQL database is...

...relational, secure, redundant  
and parallel



## The SQL language...

...is a domain-specific,  
declarative language  
...is either standalone or a glue  
language  
...has many “dialects”



## BigQuery...

...is a powerful Google  
cloud-based data warehouse  
and analytics platform  
...uses GoogleSQL for querying  
data

# In SQL, the order of clauses matters

Your SQL clauses have to be in a specific order, otherwise you'll throw an exception.

1. **SELECT** - What columns do you want?
2. **FROM** - Which table?
3. **WHERE** - What conditions must be met? \*haven't covered this yet\*
4. **GROUP BY** - What groups do you want to see?
5. **HAVING** - What conditions must be met for groups? \*haven't covered this yet\*
6. **ORDER BY** - How do you want to order the results?
7. **LIMIT** - How many rows do you want to return?

Mnemonic to remember the order: "Smelly Feet Will Give Horrible Odors, Lingeringly"

**DON'T FORGET:**

**Exit tickets**



